

OpenAI API 와 Streamlit 을 활용한 지능형 대화 에이전트 구축



1. 서론: 생성형 AI 시대의 새로운 소프트웨어 패러다임



인공지능(Artificial Intelligence) 기술의 발전, 특히 자연어 처리(NLP) 분야에서의 혁신은 소프트웨어 개발의 패러다임을 근본적으로 변화시키고 있다. 과거의 소프트웨어 엔지니어링이 명시적인 규칙(Rule)과 로직(Logic)을 코드로 구현하는 결정론적(Deterministic) 접근 방식에 의존했다면, 현재의 생성형 AI(Generative AI) 시대는 데이터와 확률 통계에 기반한 대규모 언어 모델(Large Language Model, LLM)을 활용하여 비정형적인 자연어를 이해하고 생성하는 확률론적(Probabilistic) 접근 방식을 요구한다. 이러한 변화의 중심에는 OpenAI의 GPT(Generative Pre-trained Transformer) 시리즈와 같은 파운데이션 모델(Foundation Model)이 존재하며, 이를 애플리케이션에 통합하기 위한 API(Application Programming Interface) 기술과 사용자 경험(UX)을 극대화하기 위한 인터페이스 구축 기술이 핵심 역량으로 부상하고 있다.

챗봇 개발의 전 과정을 심층적으로 분석한다. 단순히 코드를 따라 하는 수준을 넘어, LLM의 행동을 제어하는 **프롬프트 엔지니어링(Prompt Engineering)**의 인지과학적 원리, RESTful API의 무상태(Stateless) 특성을 극복하고 **멀티턴(Multi-turn) 대화**를 구현하는 메모리 관리 전략, 그리고 Python 기반의 **Streamlit** 프레임워크를 활용하여 데이터 중심의 대화형 웹 인터페이스를 구축하는 아키텍처를 포괄적으로 다룬다.

텍스트 완성(Text Completion) 모델에서 대화형(Chat) 모델로의 진화 과정을 추적하고, 시스템 프롬프트가 모델의 잠재 공간(Latent Space) 탐색에 미치는 영향을 분석하며, 궁극적으로는 사용자와 상호작용하며 지속적으로 맥락을 유지하는 지능형 에이전트(Intelligent Agent)를 구현하는 실무적 지식을 제공하고자 한다.

2. LLM 과 프롬프트 엔지니어링의 이론적 심층 분석



프롬프트 엔지니어링은 LLM 의 가중치(Weights)를 직접 수정하지 않고, 자연어 형태의 입력(Prompt)을 조작하여 모델의 출력 결과를 최적화하는 기술이다. 이는 모델이 방대한 학습 데이터 속에서 사용자의 의도에 부합하는 패턴을 찾아내도록 유도하는 '인컨텍스트 러닝(In-Context Learning)'의 일환으로 해석될 수 있다.

2.1 시스템 프롬프트(System Prompt)와 페르소나(Persona) 설계

1. 제어: AI의 목소리 조율하기

페르소나 설계: 시스템 프롬프트의 힘

OpenAI API는 단순한 텍스트 완성을 넘어, `system`, `user`, `assistant` 역할(Role) 기반의 구조적 대화를 지원합니다.

이 중 `system` 역할은 대화 전체를 관통하는 '메타 규칙'으로 작용하여 모델의 성격, 어조, 제약 조건을 정의합니다.

시스템 프롬프트는 LLM이 학습 데이터 내에서 특정 개념(예: '마법 거울')과 강하게 연관된 토큰들의 생성 확률을 높여, 출력 분포를 의도대로 재조정하는 메커니즘입니다.

****system****

user
오늘 날씨 알려줘

assistant
안개 낀 런던의 오후처럼, 오늘은 조금 습하고 흐립니다. 🌧️

user
그럼 우산이 필요할까?

assistant
네, 작은 우산 하나쯤 챙기시는 게 좋겠군요, 명탐정님. 🕒

OpenAI의 Chat Completion API는 메시지를 system, user, assistant라는 세 가지 고유한 역할(Role)로 구분하여 처리한다. 이 구조는 단순한 텍스트 완성을 넘어, 대화의 구조적 맥락을 형성하는 데 필수적이다.

2.1.1 역할(Role) 기반 메시지 처리 구조

- **System Role:** 대화의 최상위 레벨에서 모델의 행동 지침, 성격(Persona), 제약 조건, 어조(Tone)를 설정한다. 이는 대화 전체를 관통하는 '메타 규칙'으로 작용한다.
- **User Role:** 사용자의 발화나 질문을 담는다.
- **Assistant Role:** 모델의 응답을 담으며, 이전 대화 내용을 모델에게 주입할 때 사용된다.

2.1.2 페르소나 부여의 인지적 메커니즘

시스템 프롬프트가 출력 분포를 바꾸는 방식

동일한 질문: "세상에서 누가 제일 아름답니?"

**마법 거울 (Magic Mirror)**

너는 백설공주 이야기 속의 마법 거울이야.

부여된 페르소나
복종적, 고어체, 판타지적

실제 출력 (예시)
"오, 여왕님. 세상에서 가장 아름답지만, 백설공주가 더욱 아름답습니다."

언어적 특징
존칭('여왕님'), 문어체 종결어미, 원작 서사 구조 인용

**조커 (Joker)**

너는 배트맨에 나오는 조커야.

부여된 페르소나
반사회적, 냉소적, 혼돈

실제 출력 (예시)
"하하하! 아름다움이라니? 그건 혼돈과 미소 뒤에 숨겨진 진실일 뿐이지."

언어적 특징
의성어('하하하'), 추상적/철학적 개념 연결, 비꼬는 어조

페르소나 설계는 단순한 '스타일' 지정이 아니라, 모델의 잠재 공간(Latent Space) 탐색 방향을 결정하는 핵심적인 제어 기술입니다.

시스템 프롬프트가 모델의 출력 분포(Output Distribution)를 어떻게 변화시키는지를 극명하게 보여준다. 동일한 질문인 "세상에서 누가 제일 아름답니?"에 대해 시스템 프롬프트를 변경했을 때의 결과는 다음과 같이 분석된다.

시스템 프롬프트 설정	부여된 페르소나	출력 결과의 특징	언어적 분석
"너는 백설공주 이야기 속의 마법 거울이야."	복종적, 고어체, 판타지적	"오, 여왕님", "백설공주가 더욱 아름답습니다."	존칭('여왕님'), 문어체 종결어미 사용, 원작 동화의 서사 구조(Narrative Structure) 인용
"너는 배트맨에 나오는 조커야."	반사회적, 냉소적, 혼돈	"하하하!", "혼돈과 미소 뒤에 숨겨진 진정한 아름다움"	의성어('하하하')의 빈번한 사용, 추상적이고 철학적인 개념 연결, 비꼬는 듯한 어조

이러한 현상은 LLM 이 학습 과정에서 습득한 개념 간의 연관성(Association)에 기인한다. "마법

거울"이라는 키워드는 학습 데이터 내의 동화 텍스트, 고풍스러운 말투, 여왕에 대한 복종 등의 토큰들과 강하게 연결되어 있다. 시스템 프롬프트로 이를 활성화(Activation)하면, 모델은 이후 생성되는 토큰의 확률 분포를 해당 문맥에 맞춰 재조정한다. 반면 "조커"라는 키워드는 배트맨 코믹스 대사, 악당의 심리, 웃음소리 등의 토큰 확률을 높인다.

Joker_in_batman.py

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9, #
    messages=[
        {"role": "system", "content": "너는 배트맨에 나오는 조커야. 조커의 악당 캐릭터에 맞게
답변해줘"},
        {"role": "user", "content": "세상에서 누가 제일 아름답니?"},
    ] #
)

print(response)
print('----') # ⑤
print(response.choices[0].message.content)
```

mirror_in_snow_white.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
load_dotenv()

client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

response=client.chat.completions.create(
    model='gpt-4o',
    temperature=0.9,
    messages=[
        {'role':'system','content': '너는 백설공주 이야기 속의 거울이야. 그 이야기 속의 마법 거울의
캐릭터에 부합하게 답변해줘.'},
        {'role':'user','content': '세상에서 누가 제일 아름답니?'}
    ]
)
print(response)
print(response.choices[0].message.content)
```

2.1.3 시스템 프롬프트 최적화 전략

성공적인 챗봇을 위한 시스템 프롬프트 최적화 전략

1. 구체적인 페르소나 정의

❌ Bad: "너는 도움이 되는 어시스턴트야."

✅ Good: "너는 10년 차 파이썬 백엔드 개발자이자 기술 블로거야. 복잡한 개념을 초심자도 이해하기 쉽게 비유를 들어 설명해줘."

2. 명확한 출력 형식 지정

응답의 구조를 명시적으로 제어하여 후속 처리(Parsing)를 용이하게 만듭니다.

"[요약], [장점], [단점] 세 가지 섹션으로 나누어 볼렛 포인트로 정리해줘."

"반드시 RFC 8259 표준을 준수하는 JSON 형식으로만 출력해."

3. 제약 조건 설정 (Set Negative Constraints)

허용되지 않는 행동을 명시하여 환각(Hallucination) 현상을 억제하고 신뢰도를 높입니다.

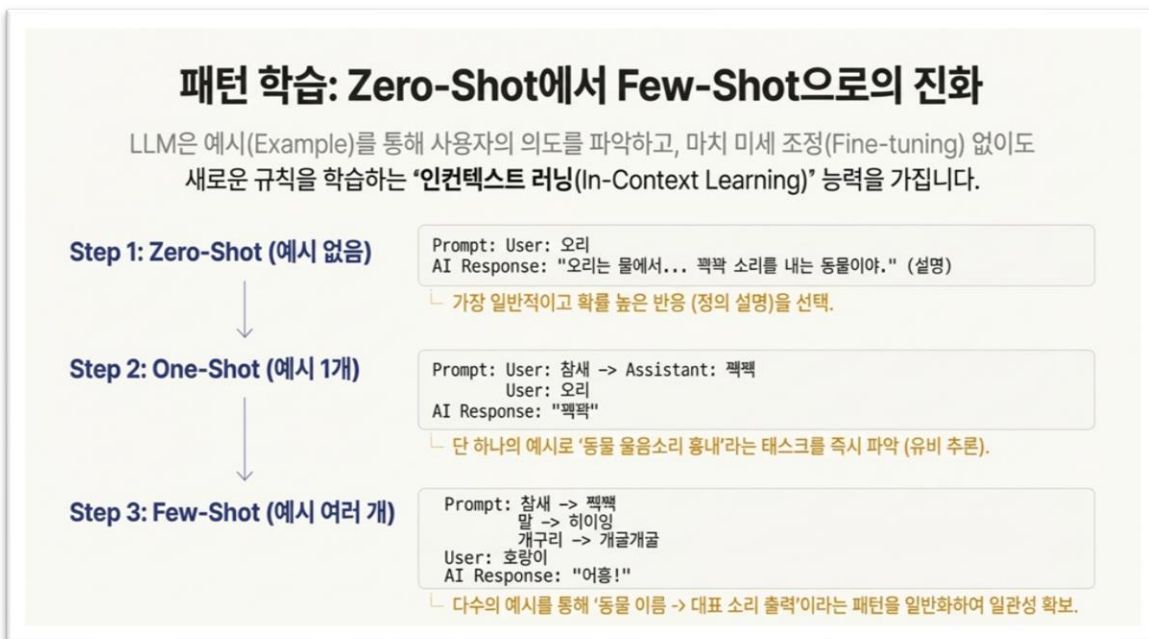
"추측해서 대답하지 마. 모르는 정보는 반드시 '확인할 수 없는 정보입니다'라고 답해."

"절대로 개인적인 의견을 제시하지 마."

성공적인 챗봇 구현을 위해서는 명확하고 구체적인 시스템 프롬프트 설계가 필수적이다. 연구 문헌들은 다음과 같은 요소를 포함할 것을 권장한다.³

1. **구체적인 페르소나 정의:** "도우미"와 같은 모호한 표현보다는 "10 년 차 파이썬 백엔드 개발자"와 같이 구체적인 전문성을 명시해야 한다.
2. **출력 형식 지정:** "볼렛 포인트로 요약해줘", "JSON 형식으로만 출력해"와 같이 응답의 구조를 제어한다.
3. **제약 조건(Negative Constraints) 설정:** "거짓말은 하지 마", "모르는 정보는 모른다고 답해"와 같은 제약을 통해 환각(Hallucination) 현상을 억제한다.

2.2 샷(Shot) 기반 프롬프팅: 제로샷에서 퓨샷으로의 확장



LLM은 방대한 텍스트 데이터로 사전 학습(Pre-training)되었기 때문에 일반적인 지식을 가지고 있지만, 사용자가 원하는 특정 형식이나 뉘앙스를 정확히 파악하기 위해서는 예시(Example)가 필요하다. 이를 샷(Shot) 기반 프롬프팅이라 하며, 예시의 수에 따라 제로샷(Zero-shot), 원샷(One-shot), 퓨샷(Few-shot)으로 분류된다.¹

2.2.1 제로샷(Zero-shot) 프롬프팅: 일반 지식의 활용

제로샷은 예시 없이 지시문(Instruction)과 입력 데이터(Input)만을 제공하는 방식이다. "오리"라는 단어를 입력했을 때, 모델은 "오리는 물에서 둥둥 떠다니고 꺽꺽 소리 내는 동물이야"라고 설명적인 답변을 내놓는다. 이는 모델이 "유치원생처럼 답변해줘"라는 시스템 프롬프트 외에 별도의 예시를 받지 못했기 때문에, '오리'라는 단어에 대한 가장 일반적이고 확률 높은 반응(정의 설명)을 선택했기 때문이다.¹

- **적용 분야:** 일반 상식 질문, 감성 분석, 요약 등 모델이 이미 잘 알고 있는 태스크에 적합하다.

01_no_prompting.py

```
from openai import OpenAI
from dotenv import load_dotenv
```

```

import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원 학생이야. 유치원생처럼 답변해줘."},
        {"role": "user", "content": "오리"},
    ]
)

print(response)
print('----')
print(response.choices[0].message.content)
# 짹짹! 오리는 물에서 헤엄도 치고, 땅에서도 걸어다니고,
# 하늘도 날아다니는 동물이야.
# 오리 너무 귀엽지 않아? 물장구 칠 때 너무 재미있어 보여! 🐼

```

2.2.2 원샷(One-shot) 프롬프팅: 의도의 명확화

원샷 프롬프팅은 단 하나의 예시를 제공하여 모델에게 태스크의 성격을 규정해주는 기법이다.

- **입력 예시:** User: 참새 -> Assistant: 짹짹
- **테스트 입력:** User: 오리
- **결과:** Assistant: 짹짹

단 하나의 예시("참새-짹짹")를 통해 모델은 사용자의 의도가 '동물에 대한 설명'이 아니라 '동물의 울음소리 흉내'임을 파악한다. 이는 모델의 추론 방향을 급격하게 수정하는 강력한 트리거 역할을

한다. 인간의 대화에서 "A 는 B 야. 그럼 C 는?"이라고 물었을 때 D 라고 유추하는 유비 추론(Analogical Reasoning)과 유사하다.

02_one_shot.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원 학생이야. 유치원생처럼 답변해줘."},
        {"role": "user", "content": "참새"},
        {"role": "assistant", "content": " 짹짹"},
        {"role": "user", "content": "오리"},
    ]
)

print(response)

print('----')
print(response.choices[0].message.content)
# ----
# 짹 짹!
```

2.2.3 퓨샷(Few-shot) 프롬프팅: 패턴 학습과 일반화

원샷 프롬프팅은 강력하지만, 예외적인 케이스나 복잡한 규칙을 처리하기에는 부족할 수 있다. 원샷 상태로 "호랑이"나 "뱀"을 물었을 때 모델은 적절한 의성어를 찾지 못해 "어흥! 무서워!"와 같이 일관성 없는 답변을 내놓았다. 이를 해결하기 위해 퓨샷 프롬프팅이 도입된다.

- 추가 예시:
 - 참새 -> 짹짹
 - 말 -> 히이잉
 - 개구리 -> 개굴개굴
- **결과 향상:** 다수의 예시를 접한 모델은 "동물의 이름이 입력되면 그 동물의 대표적인 소리를 짧게 출력해야 한다"는 규칙을 더 확고하게 학습한다. 결과적으로 "호랑이"에 대해 "어흥!", "뱀"에 대해 "슈슈슈"와 같은 답변을 생성하게 된다.¹

02_few_shot.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원 학생이야. 유치원생처럼 답변해줘."},
        {"role": "user", "content": "참새"},
        {"role": "assistant", "content": "짹짹"},
        {"role": "user", "content": "말"},
```

```

{"role": "assistant", "content": "히이잉"},
{"role": "user", "content": "개구리"},
{"role": "assistant", "content": "개굴개굴"},
{"role": "user", "content": "뱀"},
]
)

```

```

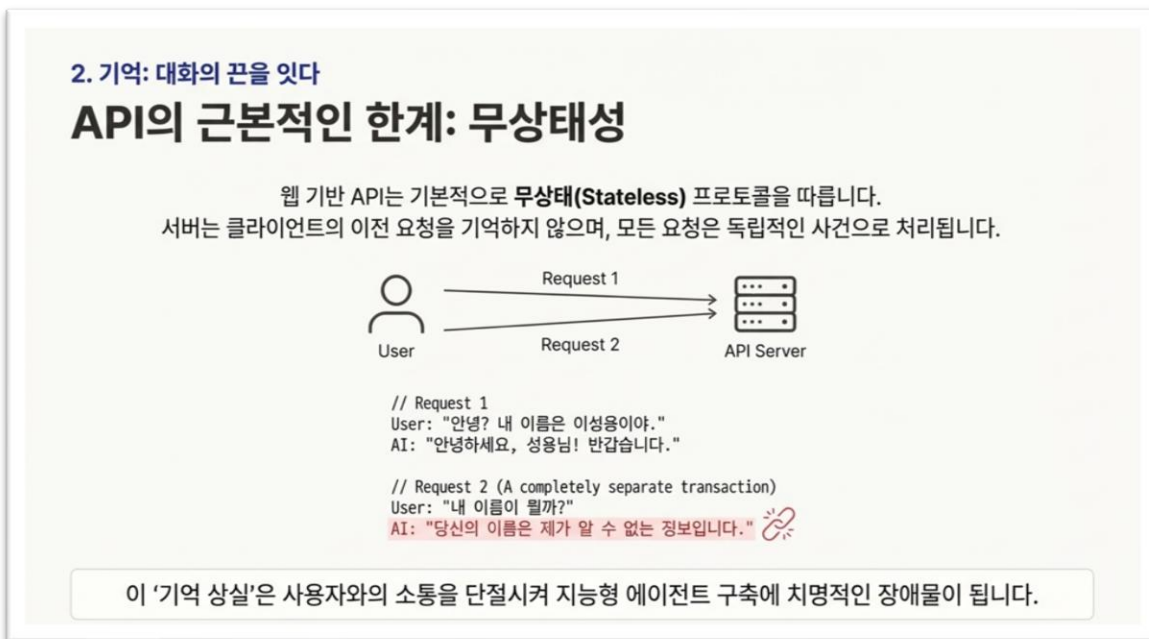
print(response)
print('----')
print(response.choices[0].message.content)
# ----
# 스크립!

```

퓨샷 프롬프팅은 모델의 파라미터를 업데이트하지 않으면서도(No Gradient Descent), 마치 미세 조정(Fine-tuning)을 한 것과 유사한 성능 향상을 이끌어낸다. 연구에 따르면 2 개에서 5 개 정도의 명확하고 다양한 예시를 제공하는 것이 비용 대비 성능 효율이 가장 뛰어나다.⁹

프롬프팅 기법	정의	메커니즘	장점	단점
Zero-shot	예시 없음	사전 학습 지식 의존	토큰 비용 최소화, 범용성	복잡한 형식 준수 어려움, 환각 가능성
One-shot	1 개 예시	태스크 정의(Task Definition)	의도 파악 속도 향상	예외 상황 처리 미흡
Few-shot	다수 예시	패턴 인식(Pattern Recognition)	높은 정확도, 일관된 스타일 유지	프롬프트 길이 증가로 인한 비용 및 속도 저하

3. OpenAI API 의 구조와 멀티턴 대화 매커니즘



인간의 대화는 연속적이다. 앞선 발언이 뒤따르는 발언의 맥락(Context)이 된다. 하지만 OpenAI 의 API, 그리고 대부분의 웹 기반 API 는 기본적으로 **무상태(Stateless)** 프로토콜을 따른다. 이는 서버가 클라이언트의 이전 요청 상태를 저장하지 않는다는 의미이다. 따라서 챗봇이 '기억'을 가지게 하려면 클라이언트 측에서 특별한 처리가 필요하다.

3.1 API 의 무상태성과 기억 상실 문제

강의 자료 ¹ 의 single_turn.py 예제는 API 의 무상태성을 시각적으로 보여준다.

1. **사용자:** "안녕? 내 이름은 해피데이야."
2. **AI:** "안녕하세요, 해피데이님! 만나서 반갑습니다."
3. **사용자:** "내 이름이 뭐까?"
4. **AI:** "당신의 이름은 제가 알 수 없는 정보입니다."

사용자가 1 번 턴에서 이름을 말했음에도 불구하고, 3 번 턴에서 AI 는 이를 기억하지 못한다. 이는 3 번 요청을 보낼 때 1 번 요청의 정보가 API 서버로 전송되지 않았기 때문이다. 모델은 매 요청을 완전히 새로운 독립적인 사건으로 취급한다.

01_single_turn.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
```

```
client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성
```

```
while True:
```

```
    user_input = input("사용자: ")
```

```
    if user_input == "exit":
```

```
        break
```

```
    response = client.chat.completions.create(
```

```
        model="gpt-4o",
```

```
        temperature=0.9,
```

```
        messages=[
```

```
            {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
```

```
            {"role": "user", "content": user_input},
```

```
        ],
```

```
    )
```

```
    print("AI: " + response.choices[0].message.content)
```

```
# 사용자: 안녕 내 이름은 해피데이야
```

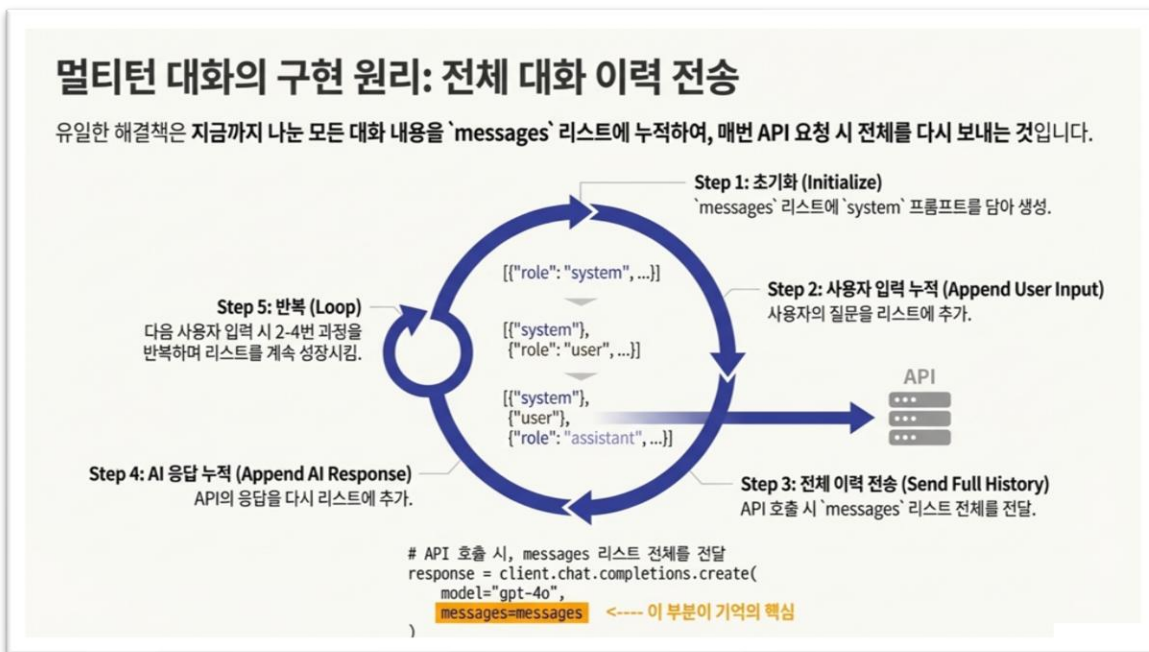
```
# AI: 안녕하세요, 해피데이님! 만나서 반갑습니다. 오늘 어떻게 도와드릴까요?
```

```
# 사용자: 내 이름이 머지?
```

```
# AI: 죄송하지만, 제가 사용자의 이름을 알 수는 없습니다. 혹시 다른 도움이 필요하시면 말씀해 주세요!
```

```
# 사용자:
```

3.2 멀티턴 대화(Multi-turn Conversation) 구현 원리



멀티턴 대화를 구현하는 유일한 방법은 지금까지 나온 모든 대화 내용을 매번 API에 다시 보내는 것이다. 이를 위해 messages라는 리스트(List) 자료구조를 사용하여 대화 이력을 누적 관리한다.¹

3.2.1 multi_turn.py 코드의 논리적 분석

강의 자료의 multi_turn.py 코드는 다음과 같은 순환 구조(Loop)를 통해 기억을 구현한다.

대화 저장소 초기화:

```
MESSAGES = [{"ROLE": "SYSTEM", "CONTENT": "너는 사용자를 도와주는 상담사야."}]
```

프로그램 시작 시, 시스템 프롬프트가 담긴 리스트를 생성한다. 이것이 대화의 '씨앗'이 된다.

1. 사용자 입력 처리 및 누적:

사용자가 입력을 하면, 이를 {"role": "user", "content": "..."} 형태의 딕셔너리로 변환하여 messages 리스트에 append 한다. 이제 messages 리스트에는 시스템 프롬프트와 현재

사용자의 질문이 담겨 있다.

2. 전체 이력 전송:

```
response = client.chat.completions.create(  
    model="gpt-4o",  
    messages=messages # 전체 리스트 전송  
)
```

API 를 호출할 때, 방금 추가된 질문뿐 만 아니라 messages 리스트 전체를 전달한다. 모델은 이 리스트를 처음부터 끝까지 읽으며 문맥을 파악한다.

3. AI 응답 누적:

API 로부터 받은 응답 역시 {"role": "assistant", "content": "..."} 형태로 변환하여 messages 리스트에 추가한다. 이제 리스트에는 [시스템 설정 -> 사용자 질문 -> AI 답변]의 순서로 데이터가 쌓인다.

4. 반복(Loop):

다음 질문이 들어오면 이 과정이 반복된다. 리스트는 [시스템 -> 질문 1 -> 답변 1 -> 질문 2]가 되고, 이를 전송받은 모델은 질문 1 과 답변 1 의 내용을 참조하여 질문 2 에 대한 답변을 생성할 수 있게 된다. 이것이 "미국 연예인 누구야?"라는 질문 후 "한국은?"이라고만 물어도 문맥을 이해하는 원리이다.

02_multi_turn.py

```
from openai import OpenAI # 오픈 AI 라이브러리를 가져오기  
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기  
  
client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성  
  
def get_ai_response(messages):  
    response = client.chat.completions.create(  
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
```

```

    temperature=0.9, # 응답 생성에 사용할 temperature 설정
    messages=messages, # 대화 기록을 입력으로 전달
)
return response.choices[0].message.content # 생성된 응답의 내용 반환

messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
]

while True:
    user_input = input("사용자: ") # 사용자 입력 받기

    if user_input == "exit": # 사용자가 대화를 종료하려는지 확인인
        break

    messages.append({"role": "user", "content": user_input}) # 사용자 메시지를 대화 기록에 추가
    ai_response = get_ai_response(messages) # 대화 기록을 기반으로 AI 응답 가져오기
    messages.append({"role": "assistant", "content": ai_response}) # AI 응답 대화 기록에 추가하기

    print("AI: " + ai_response) # AI 응답 출력
# 사용자: 안녕 내 이름은 해피데이야
# AI: 안녕하세요, 해피데이님! 만나서 반갑습니다. 어떻게 도와드릴까요?
# 사용자: 내 이름이 머지?
# AI: 당신의 이름은 "해피데이"라고 말씀하셨습니다. 맞나요?
# 사용자:

```

3.3 컨텍스트 윈도우(Context Window)와 토큰 관리 전략

끝나지 않는 대화의 비용: 컨텍스트 윈도우와 토큰 관리

대화가 길어지면 `messages` 리스트가 무한정 커질 수 없습니다. LLM은 한 번에 처리할 수 있는 토큰의 양, 즉 **컨텍스트 윈도우(Context Window)** (예: GPT-4o, 128k 토큰)에 물리적 한계가 있으며, 입력 토큰은 곧 비용입니다.

전략 (Strategy)	설명 (Description)	장점 (Pros)	단점 (Cons)
전체 대화 유지 (Buffer Memory)	모든 대화 내용을 그대로 유지	완벽한 문맥 보존	비용 급증, 토큰 한계 도달
슬라이딩 윈도우 (Window Memory)	최근 N개의 대화만 유지하고 오래된 것은 삭제	토큰 사용량 일정 유지	오래된 중요 정보 망각
대화 요약 (Summary Memory)	오래된 대화를 LLM을 이용해 요약문으로 변환	긴 문맥을 압축적으로 유지	요약 시 디테일 손실, 추가 비용

상용 서비스에서는 반드시 토큰 수를 계산하고, 서비스의 특성에 맞는 메모리 삭제(Trimg) 전략을 도입해야 합니다.

멀티턴 대화를 위해 무한정 데이터를 쌓을 수는 없다. LLM 은 한 번에 처리할 수 있는 정보의 양, 즉 **컨텍스트 윈도우(Context Window)**에 물리적인 제한이 있다 (예: GPT-4o 는 128k 토큰). 또한, 입력 토큰의 양은 비용과 직결된다.

따라서 실무에서는 다음과 같은 메모리 관리 전략이 필수적이다.

예제는 가장 기본적인 '전체 대화 유지' 방식을 사용하고 있으나, 실제 상용 서비스를 구축할 때는 반드시 토큰 수를 계산하고 적절한 삭제(Trimming) 전략을 도입해야 한다.

4. Streamlit 을 활용한 대화형 웹 인터페이스 구축

3. 상호작용: 사용자에게 다가가기

Streamlit의 힘과 함정: Rerun 메커니즘

Streamlit을 사용하면 프론트엔드 기술 없이 순수 Python만으로 빠르고 직관적인 대화형 웹 UI를 구축할 수 있습니다. Streamlit의 핵심 실행 모델은 사용자의 모든 상호작용(버튼 클릭, 텍스트 입력)마다 **스크립트 전체를 처음부터 끝까지 다시 실행(Rerun)**하는 것입니다.

백엔드에서 구현한 '기억'을 이 Rerun 환경에서 어떻게 보존할 수 있을까요?

Streamlit's Rerun Mechanism & The Memory Problem

터미널(Terminal) 창에서의 텍스트 기반 인터페이스(CLI)는 개발자에게는 효율적일지 모르나, 최종 사용자에게는 접근성이 매우 낮다. 이를 개선하기 위해 Python 기반의 웹 애플리케이션 프레임워크인 **Streamlit** 을 사용하여 현대적인 챗봇 UI 를 구축한다.

4.1 Streamlit 의 아키텍처와 Session State 의 중요성

Streamlit 은 데이터 과학자와 AI 엔지니어가 HTML, CSS, JavaScript 와 같은 프론트엔드 기술 없이도 웹 앱을 만들 수 있게 해주는 강력한 도구이다. 하지만 Streamlit 의 ****실행 모델(Execution Model)****을 이해하지 못하면 챗봇 구현에 실패하기 쉽다.

4.1.1 리런(Rerun) 메커니즘

Streamlit 은 사용자가 버튼을 클릭하거나 텍스트를 입력하는 등 상호작용(Interaction)이 발생할 때마다 ****파이썬 스크립트 전체를 처음부터 끝까지 다시 실행(Rerun)****한다.¹⁴ 이는 코드를 단순히 만들지만, 변수에 저장된 데이터가 매번 초기화된다는 문제를 야기한다. 예를 들어, 대화 내용을 일반 리스트 변수 `chat_history =`에 저장하면, 사용자가 새 메시지를 입력할 때마다 스크립트가 리런되면서 `chat_history` 가 다시 빈 리스트로 초기화되어 버린다.

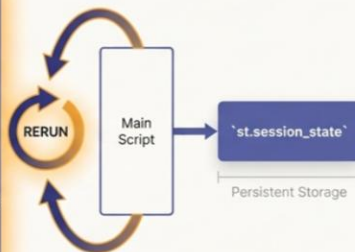
4.1.2 해결책: st.session_state

영속성의 열쇠: st.session_state

솔루션: Streamlit은 **Rerun**이 발생해도 데이터가 사라지지 않는 특별한 딕셔너리 형태의 저장 공간인 `st.session_state`를 제공합니다. 챗봇의 대화 기록(`messages`)은 반드시 이곳에 저장해야 합니다.

```
# 앱이 최초 실행될 때만 messages 리스트를 초기화
# 이 조건문 덕분에 Rerun 시에는 기존 대화가 유지됩니다.

if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "assistant", "content": "How can I help you?"}
    ]
```



이 문제를 해결하기 위해 Streamlit은 `st.session_state`라는 영구 저장소(Persistence Layer)를 제공한다.¹ `st.session_state`는 브라우저 탭이 열려 있는 동안 유지되는 딕셔너리 형태의 저장 공간이다. 리런이 발생하더라도 이곳에 저장된 데이터는 사라지지 않는다. 챗봇의 대화 기록(messages)은 반드시 이곳에 저장되어야 한다.

4.2 대화형 UI 컴포넌트 심층 분석

Streamlit은 챗봇 구현을 위한 직관적인 고수준(High-level) 함수들을 제공한다. 강의 자료 ¹의 코드를 바탕으로 각 컴포넌트의 기능을 분석한다.

4.2.1 st.chat_message

이 함수는 채팅 인터페이스의 시각적 요소를 담당한다.

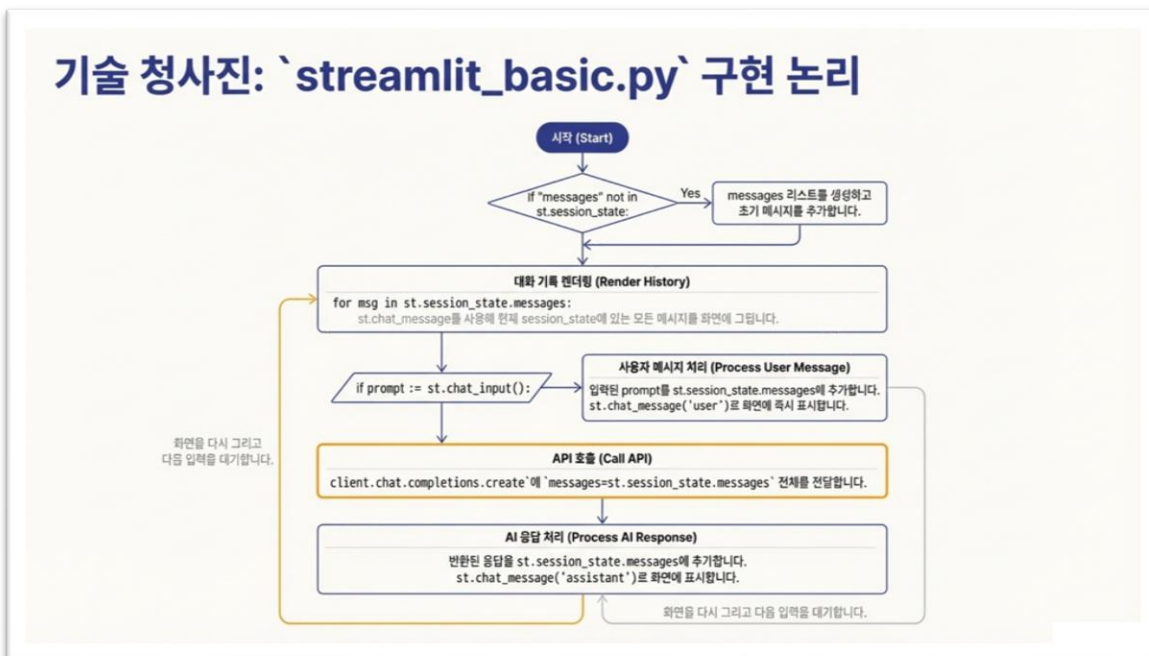
- **기능:** 사용자(user)와 AI(assistant)의 메시지를 구분하여 보여주는 컨테이너를 생성한다.
- **특징:** role 매개변수에 따라 아이콘과 메시지의 정렬(좌측/우측)을 자동으로 처리한다. with 구문과 함께 사용하여 텍스트뿐만 아니라 차트, 데이터프레임, 이미지 등 다양한 미디어를 채팅 말풍선 안에 포함시킬 수 있다.¹⁶

4.2.2 st.chat_input

이 함수는 사용자의 입력을 받는 인터페이스를 담당한다.

- **기능:** 화면 하단에 고정된 텍스트 입력창을 생성한다.
- **작동 방식:** 사용자가 엔터 키를 누르면 입력된 텍스트를 반환하고, True 로 평가되어 조건문 내부의 코드를 실행한다. 동시에 스크립트 리런을 트리거(Trigger)한다.¹⁷

4.3 streamlit_basic.py 코드의 구현 논리 분석



`streamlit_basic.py` 는 앞서 설명한 모든 개념이 통합된 완성형 코드이다. 이를 라인 단위의 흐름으로 분석하면 다음과 같다.

1. 환경 설정 및 API 클라이언트 초기화:

`dotenv` 를 통해 보안이 중요한 API 키를 환경 변수에서 로드한다. 이는 코드 내에 키를 하드코딩하지 않는 보안 모범 사례(Best Practice)를 따르는 것이다.

2. 세션 상태 초기화:

```
if "messages" not in st.session_state:
```

```
    st.session_state["messages"] = [{"role": "assistant", "content": "How can I help you?"}]
```

앱이 최초로 실행될 때만 messages 리스트를 초기화한다. session_state 를 체크하는 조건문(if... not in...) 덕분에 리런 시에는 초기화 코드가 실행되지 않고 기존 대화가 유지된다.

3. 대화 기록 렌더링 (Rendering):

```
for msg in st.session_state.messages:
```

```
    st.chat_message(msg["role"]).write(msg["content"])
```

이 부분은 챗봇 UI 의 핵심이다. 스크립트가 실행될 때마다 session_state 에 저장된 과거의 모든 메시지를 순회하며 화면에 다시 그려준다. 이 코드가 없다면 사용자는 가장 최근의 메시지만 보게 될 것이다.¹⁴

4. 사용자 인터랙션 루프:

if prompt := st.chat_input(): 구문은 사용자의 입력을 대기한다. 입력이 발생하면:

- **사용자 메시지 처리:** 입력된 prompt 를 session_state 에 추가하고 st.chat_message("user")로 화면에 즉시 출력한다.
- **API 호출:** client.chat.completions.create 함수에 messages=st.session_state.messages 를 전달하여 멀티턴 문맥을 포함한 요청을 보낸다.
- **AI 응답 처리:** 반환된 응답을 session_state 에 추가하고 st.chat_message("assistant")로 화면에 출력한다.

이러한 구조를 통해 사용자는 웹 브라우저 상에서 깔끔한 UI 를 통해 AI 와 자연스럽게 대화할 수 있으며, 내부적으로는 세션 상태 관리를 통해 대화의 맥락이 끊기지 않고 유지된다.

전체 소스

streamlit_basic.py

```
import streamlit as st
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

```
# (0) 사이드바에서 api_key 입력하는 부분
```

```

with st.sidebar:
    openai_api_key = os.getenv('OPENAI_API_KEY')
    # openai_api_key = st.text_input("OpenAI API Key", key="chatbot_api_key", type="password")
    "[Get an OpenAI API key](https://platform.openai.com/account/api-keys)"
    "[View the source code](https://github.com/streamlit/llm-examples/blob/main/Chatbot.py)"
    "[![Open in GitHub Codespaces](https://github.com/codespaces/badge.svg)](https://codespaces.new/streamlit/llm-examples?quickstart=1)"

st.title("💬 Chatbot")

# (1) st.session_state 에 "messages"가 없으면 초기값을 설정
if "messages" not in st.session_state:
    st.session_state["messages"] = [{"role": "assistant", "content": "How can I help you?"}]

# (2) 대화 기록을 출력
for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"])

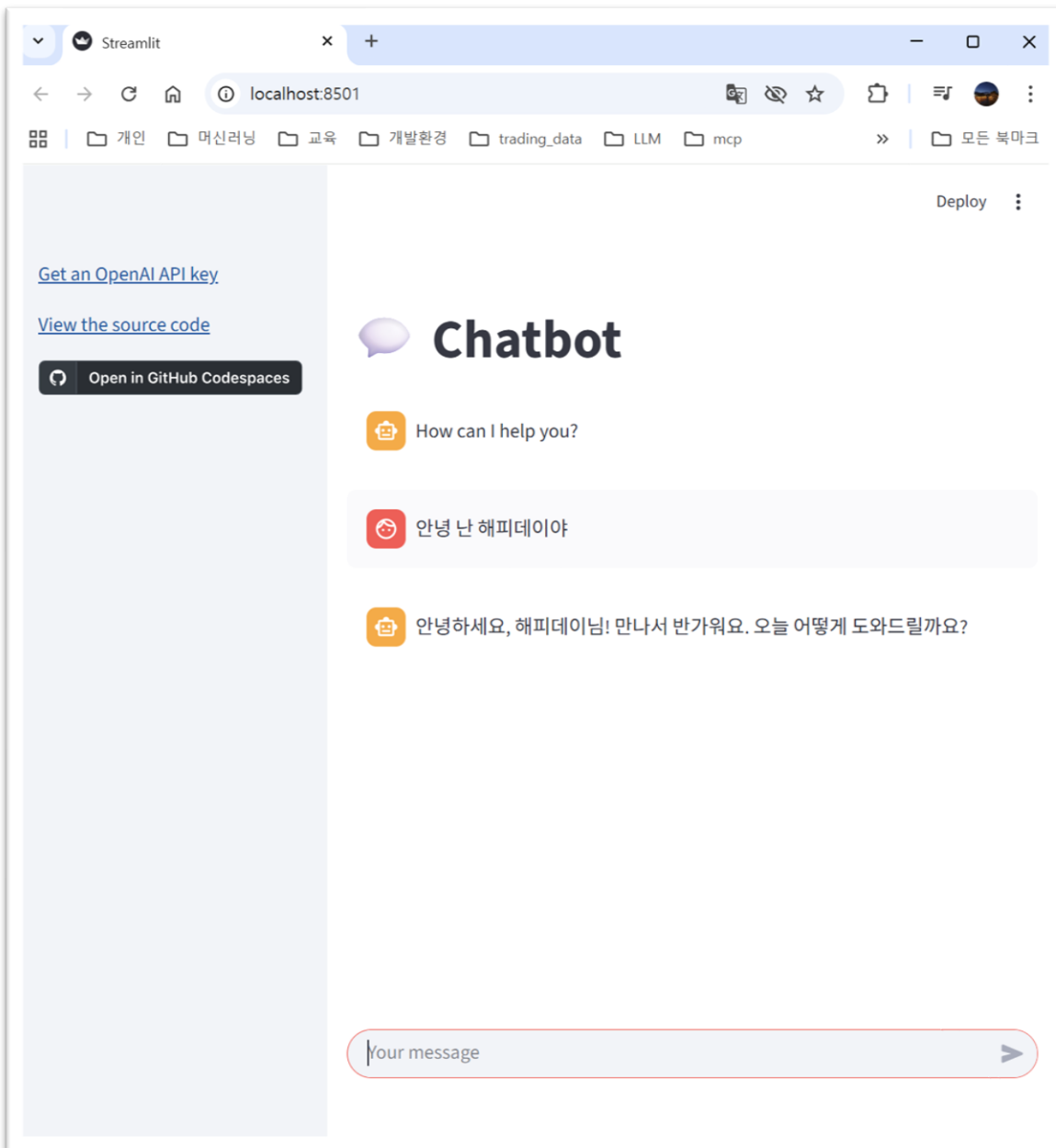
# (3) 사용자 입력을 받아 대화 기록에 추가하고 AI 응답을 생성
if prompt := st.chat_input():
    if not openai_api_key:
        st.info("Please add your OpenAI API key to continue.")
        st.stop()

    client = OpenAI(api_key=openai_api_key)
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)
    response = client.chat.completions.create(model="gpt-4o",
    messages=st.session_state.messages)
    msg = response.choices[0].message.content
    st.session_state.messages.append({"role": "assistant", "content": msg})
    st.chat_message("assistant").write(msg)

```

실행코드

streamlit run streamlit_basic.py



5. 결론 및 향후 발전 방향

OpenAI API 와 Streamlit 을 활용하여 대화형 AI 에이전트를 구축하는 전 과정을 이론과 실무의 관점에서 포괄적으로 분석하였다.

첫째, **프롬프트 엔지니어링**은 단순한 질문 작성이 아니라 모델의 인지적 행동을 설계하는 과정임을 확인하였다. 시스템 프롬프트를 통한 페르소나 설정은 모델의 답변 스타일과 지식의 범위를 규정하며, 퓨샷 프롬프팅은 예시를 통해 모델의 추론 패턴을 실시간으로 학습시키는 강력한 도구이다.

둘째, **멀티턴 대화**의 구현은 API 의 무상태성을 극복하기 위한 클라이언트 측의 메모리 관리 전략에 달려 있다. 대화 이력을 누적하여 전송하는 방식은 문맥을 유지하는 가장 확실한 방법이지만, 토큰 제한과 비용 문제를 고려한 최적화 전략이 수반되어야 한다.

셋째, **Streamlit** 은 데이터 중심의 Python 개발자가 복잡한 웹 기술 없이도 직관적인 챗봇 UI 를 구현할 수 있게 해주는 핵심 솔루션이다. Session State 를 활용한 상태 관리는 리런 기반의 실행 모델을 가진 Streamlit 에서 데이터 지속성을 보장하는 유일한 방법이다.

향후 발전 방향 (Future Work)

구축한 챗봇은 기초적인 수준에 불과하며, 상용 수준의 서비스로 발전시키기 위해서는 다음과 같은 심화 기술의 도입이 필요하다.

- **RAG (Retrieval-Augmented Generation):** 벡터 데이터베이스(Vector DB)를 활용하여 모델이 학습하지 않은 외부 데이터나 기업 내부 문서를 검색하고, 이를 바탕으로 답변하게 하는 기술.
- **Function Calling & Tool Use:** 챗봇이 단순히 텍스트만 생성하는 것이 아니라, 날씨 API 를 호출하거나 주식 매매 주문을 넣는 등 실제 세계와 상호작용하는 액션(Action)을 수행하게 하는 기술.¹⁸
- **LangChain & LangGraph:** 복잡한 대화 흐름, 메모리 관리, 프롬프트 템플릿 등을 보다 체계적으로 관리하기 위한 오케스트레이션(Orchestration) 프레임워크의 활용.¹⁹

이러한 기술적 요소들을 단계적으로 학습하고 적용함으로써, 단순한 챗봇을 넘어선 고도의 지능형 에이전트(Intelligent Agent) 개발이 가능할 것이다. 본 보고서가 그러한 여정의 견고한 디딤돌이 되기를 기대한다.